Comparative Analysis of Real Estate Prediction Models on Property24 Listings

Lathitha Nongauza

***Abstract*—This paper presents a comparative analysis of four machine learning models—deep neural networks, random forests, support vector machines, and XGBoost—applied to the task of predicting the South African province of origin for property listings scraped from Property24. We investigate the effectiveness of standard property features (price, bedrooms, bathrooms, size, garage count) for geographic classification and identify fundamental limitations in price-based feature spaces where provinces with similar price-per-square-meter distributions become indistinguishable. To address this limitation, we introduce a novel positional encoding methodology that augments property features with spatially sampled coordinates derived from province-specific Gaussian distributions. Experimental results demonstrate that while positional encoding dramatically improves classification performance to near-perfect accuracy, it introduces significant overfitting whereby models learn to memorise location-specific patterns rather than meaningful property-location relationships. This work highlights critical considerations for feature engineering in geographic classification tasks and provides insights into the trade-offs between discriminatory power and representational learning.**

I. Introduction

The South African real estate market presents unique challenges and opportunities for predictive modelling. With nine distinct provinces exhibiting varied economic conditions, demographic patterns, and property price structures, accurately classifying property listings by geographic origin has significant implications for market analysis, valuation models, and real estate analytics platforms. However, the relationship between property attributes and geographic location is complex and often confounded by overlapping price distributions across provinces.

This study addresses the question: Can machine learning models reliably predict a property's province of origin using only its structural and price-based features? We investigate this problem through a comparative analysis of four fundamentally different modelling approaches: deep neural networks, random forests, support vector machines, and XGBoost. Each model class offers distinct advantages—neural networks excel at capturing complex non-linear relationships, random forests provide robust ensemble-based predictions, SVMs offer maximum-margin classification with kernel-based non-linearity, and XGBoost delivers state-of-the-art gradient boosting performance.

Our methodology encompasses the complete machine learning pipeline: custom web scraping of Property24 listings on the 26th of January 2026, rigorous data cleaning and preprocessing, feature engineering including price-per-square-meter ratios, and a novel positional encoding framework that augments property features with spatially sampled coordinates. This positional encoding, based on province-specific Gaussian distributions and shear transformations, enables us to simulate geographic information absent from the raw listing data.

The key contributions of this work are threefold. First, we provide a systematic evaluation of four major model classes on a real-world geographic classification task, identifying their relative strengths and limitations. Second, we demonstrate the fundamental inadequacy of price-based features alone for province classification, revealing that provinces with similar price-per-square-meter distributions are inherently indistinguishable in such feature spaces. Third, we introduce and critically evaluate a positional encoding methodology, showing that while it achieves near-perfect classification accuracy, it introduces problematic overfitting whereby models prioritise spatial location over meaningful property-attribute relationships.

Our results reveal important insights for practitioners: the choice of feature representation fundamentally determines model behaviour, and high accuracy achieved through spatial features may mask underlying failures in learning transferable representations. These findings have implications beyond real estate analytics, informing best practices for geographic classification tasks in domains where spatial information is available but may dominate the learning signal.

The remainder of this paper is organised as follows. Section II provides the mathematical background for each model architecture. Section III details our data acquisition and preprocessing methodology. Section IV presents our positional encoding framework. Section V describes the specific model implementations and hyperparameters. Section VI presents comparative results. Section VII discusses the implications of our findings, and Section VIII concludes with recommendations for future work.

II. Background

A. Neural Networks

A feedforward neural network learns a nonlinear mapping $f : \mathbb{R}^d \to \{1, \ldots, 9\}$ by composing linear transformations and activation functions.

1) Linear Layer: Each layer computes $\mathbf{h} = W\mathbf{x} + \mathbf{b}$, where W is a weight matrix and $\mathbf{b}$ a bias vector.

2) Activation Function: The ReLU activation $\mathrm{ReLU}(z) = \max(0, z)$ introduces nonlinearity.

3) Batch Normalization: For a mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$, batch norm normalises each feature

to zero mean and unit variance, then applies learnable scale and shift parameters γ, β:

$$y_i = \gamma \hat{x}_i + \beta.$$

During inference, running statistics accumulated during training are used.

4) Dropout: During training, each neuron is independently zeroed with probability p and the remaining activations are scaled by $1/(1-p)$ to preserve expected value. Dropout is disabled at inference.

5) Cross-Entropy Loss: For multi-class classification with K classes, the loss measures the divergence between predicted probabilities (softmax of logits) and the true label:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbf{1}[y_i = k] \log \hat{p}_{i,k}.$$

6) Adam Optimizer: Adam maintains per-parameter adaptive learning rates using estimates of first and second moments of gradients.

7) L2 Regularization: An L2 penalty $\lambda \|\theta\|^2$ is added to the loss to discourage large weights.

8) Gradient Clipping: If the global gradient norm exceeds a threshold τ, gradients are scaled down to have norm τ while preserving direction.

9) Prediction: At inference, the predicted class is the argmax of the logits, and probabilities are obtained via softmax:

$$\hat{y} = \arg\max_k z_k, \quad \hat{p}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

B. Random Forest

Random Forest is an ensemble of decision trees trained on bootstrap samples.

1) Decision Tree: A decision tree recursively partitions the feature space. At each node, it selects a feature j and threshold t that maximize impurity reduction:

$$\Delta I = I(\mathcal{D}) - \frac{|\mathcal{D}_L|}{|\mathcal{D}|} I(\mathcal{D}_L) - \frac{|\mathcal{D}_R|}{|\mathcal{D}|} I(\mathcal{D}_R),$$

where I is the impurity measure. Random Forest uses Gini impurity:

$$I_G(\mathcal{D}) = 1 - \sum_{k=1}^{K} p_k^2,$$

with p_k the proportion of class k in node $\mathcal{D}$.

2) Bootstrap Sampling: Each tree is trained on a bootstrap sample of the original dataset, drawn with replacement. About 63.2% of unique samples appear in each bootstrap sample; the remaining out-of-bag samples provide an internal error estimate.

3) Random Feature Subsampling: At each split, only a random subset of m features (typically $\lfloor \sqrt{d} \rfloor$) is considered, decorrelating the trees.

4) Prediction: For a new sample, each tree votes for a class; the final prediction is the majority vote.

C. Support Vector Machine

An SVM finds a hyperplane that maximises the margin between classes, extended to nonlinear boundaries via the kernel trick.

1) Hard-Margin SVM: For linearly separable data, the SVM finds a hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximizes the margin $2/\|\mathbf{w}\|$ by solving:

$$\min_{\mathbf{w},b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

2) Soft-Margin SVM: For non-separable data, slack variables ξ_i allow margin violations:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i, \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \ \xi_i \geq 0.$$

The parameter C controls the trade-off between margin width and violations.

3) Kernel Trick: The kernel function $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ computes inner products in a high-dimensional feature space without explicitly constructing ϕ. The RBF kernel is:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right).$$

The parameter γ determines the influence radius of each training point.

4) Multi-Class Extension: For K classes, one-vs-one (OvO) trains $K(K-1)/2$ binary classifiers; the class with the most votes wins. Platt scaling can be applied to obtain calibrated probabilities.

5) Decision Function: The prediction for a new point $\mathbf{x}$ is:

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i \in \mathcal{S}} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b\right),$$

where $\mathcal{S}$ is the set of support vectors ($\alpha_i > 0$).

D. XGBoost

XGBoost builds an additive ensemble of B trees sequentially, where each new tree corrects the residuals of the previous ensemble.

1) Additive Model: At round t, the prediction for sample i is $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)$, starting from $\hat{y}_i^{(0)} = 0$.

2) Regularized Objective: At each round, a new tree f_t minimises:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} \ell\left(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) + \Omega(f_t),$$

where ℓ is the loss function (e.g., multiclass log-loss) and $\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$ penalises tree complexity (T leaves, leaf weights w_j). The objective is approximated by a second-order Taylor expansion.

3) Split Finding: For each candidate split, the gain (loss reduction) is computed; a split is made only if the gain exceeds γ, providing built-in pruning.

4) Loss Function: Multiclass Log-Loss: For K classes, one tree is grown per class per round. The raw scores z_k are converted to probabilities via softmax:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

The multiclass log-loss is:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbf{1}[y_i = k] \log \hat{p}_{i,k}.$$

5) Learning Rate: After computing optimal leaf weights, they are scaled by the learning rate $\eta < 1$ to leave room for future trees to refine the fit.

6) Stochastic Subsampling: Row and column subsampling (without replacement) reduces correlation between trees and improves generalisation.

7) Feature Importance: XGBoost computes feature importance as the total gain accumulated by each feature across all splits in all trees:

$$\text{Importance}(j) = \sum_{t=1}^{B} \sum_{\substack{v \in T_t \\ \text{split on } j}} \text{Gain}_v.$$

III. Data Acquisition

1) Data scraping: The data collection phase utilizes a custom-built scraping engine designed to interface with real estate listing platforms. The process is optimised for efficiency and reliability through the following mechanisms:

Multi-threaded Execution: To minimise the total time required for data retrieval, the DataCollector class implements Python's threading library. Each target URL is processed in a separate thread, allowing for concurrent HTTP requests across multiple listing pages.

Thread Safety: A threading.Lock mechanism is employed to manage access to the shared data list. This ensures that when individual threads return property data, the "append" operations do not conflict, maintaining data integrity.

Request Ethics and Robustness: To simulate human-like browsing and avoid IP rate-limiting, the system injects a randomised delay (random.uniform(3, 180)) between requests.

DOM Parsing: The Scraper class utilises BeautifulSoup to parse the HTML DOM. It specifically targets listing containers (e.g., p24 content) to extract features such as titles, prices, locations, and structural attributes (bedrooms, bathrooms, and erf size).

IV. Data Preparation

A. Data Preparation

Once the raw data is retrieved, the DataPreparator class transforms the semi-structured HTML text into a tabular format suitable for quantitative analysis:

Numerical Extraction: Raw strings often contain currency symbols, whitespace, and unit characters (e.g., "m²"). The preparation logic applies a regex-style filter to strip non-numeric characters, ensuring that fields like Price and Size sqm are stored as integers.

Feature Mapping: The extracted data is mapped into a pandas DataFrame, creating a structured feature set. This include:

Categorical: Title, Location, and Details URL.

Numerical: Price, Bed, Bath, Garage, and Size (sqm).

Data Serialisation: The final cleaned dataset is exported to a comma-separated values (CSV) format, providing a persistent and portable foundation for the subsequent statistical modelling and visualisation stages of the study.

B. Data Cleaning

Following the raw data acquisition, a rigorous multi-stage preprocessing pipeline was implemented to ensure the dataset's integrity and statistical relevance. This process transitions the raw scraping output into a refined dataset for predictive modelling.1) Data Sanitisation and Structural Cleaning. The initial phase involves noise reduction by removing non-predictive features and redundant entries. Feature Selection: Irrelevant columns, including Details URL, Title, and Location (where redundant to the Province field), are dropped to reduce dimensionality. Deduplication: Identical records—often the result of multi-threaded scraping that overlaps across paginated results—are removed to prevent biased training results. Domain Constraint Filtering: To focus the analysis on standard residential assets, the dataset is restricted to properties with a valuation between R500,000 and R100,000,000. This removes extreme outliers and non-residential anomalies.2) Missing Value Imputation and Feature Engineering. To maintain a robust sample size without sacrificing accuracy, specific strategies are employed for null or zero-value attributes: Discrete Variable Imputation: Missing values for Beds, Baths, and Garages are treated as zero-valued features, assuming a lack of information implies the absence of that specific amenity. Inferred Spatial Metrics: A critical challenge in real estate data is the absence of property size (Size sqm). To address this, a Provincial Average Price-per-Sqm heuristic is calculated. For properties with missing area metrics, the size is statistically inferred by dividing the property price by the mean price-per-square-meter of its respective province.3) Statistical Outlier Detection. To ensure the model is not skewed by "long-tail" data or entry errors, the Interquartile Range (IQR) method is applied across all primary numerical features (Price, Size, Beds, Baths, and Garages). The IQR Method: For each feature, data points falling outside the range defined by $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ are excluded. ShutterstockExploreImpact: This process normalises the distribution of the dataset, ensuring that the resulting analysis reflects the core market trends of South African real estate rather than extreme luxury or distressed asset anomalies.4) Serialisation. The final processed dataset is exported to a structured CSV format, resulting in a clean, high-fidelity foundation for the subsequent analysis phase.

V. Positional Encoding

A. Euclidean Space and Coordinate System

We define a 2D Euclidean space with the map centre at the origin (Figure 1a):

$$\mathbf{c} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Each province is represented by a 2D Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ where $\boldsymbol{\mu}$ is the province centre and $\boldsymbol{\Sigma}$ is the covariance matrix (Figure 1b) in order to preserve spatial relationships.

B. Province Means

Each province mean is defined as an offset from the map centre $\mathbf{c}$ where each offset was chosen visually from (Figure 1a):

$$\boldsymbol{\mu}_{\text{province}} = \mathbf{c} + \boldsymbol{\delta}_{\text{province}}$$

$$\begin{aligned}
\boldsymbol{\mu}_{\text{EC}} &= \mathbf{c} + \begin{pmatrix} 74.1 \\ -189.3 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{FS}} &= \mathbf{c} + \begin{pmatrix} 93.1 \\ 4.0 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{GAU}} &= \mathbf{c} + \begin{pmatrix} 159.3 \\ 129.7 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{KZN}} &= \mathbf{c} + \begin{pmatrix} 275.3 \\ -16.0 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{LP}} &= \mathbf{c} + \begin{pmatrix} 229.3 \\ 252.4 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{MP}} &= \mathbf{c} + \begin{pmatrix} 259.3 \\ 128.2 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{NW}} &= \mathbf{c} + \begin{pmatrix} 45.1 \\ 117.2 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{NC}} &= \mathbf{c} + \begin{pmatrix} -147.2 \\ -64.1 \end{pmatrix} \\
\boldsymbol{\mu}_{\text{WC}} &= \mathbf{c} + \begin{pmatrix} -193.2 \\ -261.4 \end{pmatrix}
\end{aligned}$$

C. Province Covariance Matrices

Each province has a diagonal covariance matrix $\boldsymbol{\Sigma} = \begin{pmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{pmatrix}$, reflecting isotropic (circular) uncertainty with no correlation between axes.

$$\begin{aligned}
\boldsymbol{\Sigma}_{\text{EC}} &= \begin{pmatrix} 2000 & 0 \\ 0 & 2000 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{FS}} &= \begin{pmatrix} 1500 & 0 \\ 0 & 1500 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{GAU}} &= \begin{pmatrix} 250 & 0 \\ 0 & 250 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{KZN}} &= \begin{pmatrix} 1000 & 0 \\ 0 & 1000 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{LP}} &= \begin{pmatrix} 1500 & 0 \\ 0 & 1500 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{MP}} &= \begin{pmatrix} 1000 & 0 \\ 0 & 1000 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{NW}} &= \begin{pmatrix} 1200 & 0 \\ 0 & 1200 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{NC}} &= \begin{pmatrix} 4000 & 0 \\ 0 & 4000 \end{pmatrix} \\
\boldsymbol{\Sigma}_{\text{WC}} &= \begin{pmatrix} 1000 & 0 \\ 0 & 1000 \end{pmatrix}
\end{aligned}$$

D. 2D Gaussian Probability Density Function

The probability density function for each province is:

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{2\pi\sqrt{|\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

where $\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}$ is a 2D coordinate on the map, $|\boldsymbol{\Sigma}|$ is the determinant of the covariance matrix, and $(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})$ is the squared Mahalanobis distance from $\mathbf{x}$ to the province centre.

E. Shear Transformation

A shear transformation is applied to the coordinate system via the matrix:

$$\mathbf{T} = \begin{pmatrix} 1 & 0 \\ s & 1 \end{pmatrix}$$

where s is the shear parameter. This keeps vertical lines fixed while tilting horizontal lines (Figure 2a). We change the coordinate system so that the Gaussians can more closely fit to the shape of the provinces while maintaining spatial relationships.

1) Transformation of Means: Under the shear transformation, each mean transforms as:

$$\boldsymbol{\mu}' = \mathbf{T}\boldsymbol{\mu}$$

This represents the Gaussians in a new coordinate system which preserves distances between the centres of each province (Figure 2b).

2) Transformation of Covariances: The covariance matrix transforms under a linear map $\mathbf{T}$ as:

$$\boldsymbol{\Sigma}' = \mathbf{T}\boldsymbol{\Sigma}\mathbf{T}^\top$$

F. Sampling

To assign a spatial position to a listing from province k, we sample:

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}'_k, \boldsymbol{\Sigma}'_k)$$

where $\boldsymbol{\mu}'_k$ and $\boldsymbol{\Sigma}'_k$ are the transformed mean and covariance. The sampled $\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}$ gives the listing's position in the sheared coordinate system.

G. Normalization

The raw feature matrix $X_{\text{raw}} \in \mathbb{R}^{n \times d}$ is standardized to zero mean and unit variance. This ensures no single feature dominates due to scale differences (e.g. price in hundreds of thousands vs. number of bedrooms in single digits).

H. Decorrelation

The normalised data is projected onto the eigenvectors of its covariance matrix:

$$Z = \hat{X}Q, \quad \Sigma = \frac{1}{n-1}\hat{X}^\top \hat{X} = Q\Lambda Q^\top$$

where $Q \in \mathbb{R}^{d \times d}$ contains the eigenvectors as columns and $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_d)$ contains the eigenvalues. This decorrelates the features:

$$\Sigma_Z = Q^\top \Sigma Q = \Lambda$$

The projected data Z is what is fed into the neural network as input.

I. Label Encoding

Province labels are integer-encoded via a label encoder:

$$\text{Province string} \xrightarrow{\text{LabelEncoder}} y \in \{0, 1, \ldots, 8\}$$

For CrossEntropy loss, labels are kept as integers (class indices), not one-hot vectors.

VI. Model Architecture

This section describes the specific implementation of each model used for province classification. All models receive the decorrelated features Z (or normalised features where noted) and predict one of nine provinces.

A. Train/Test Split

The dataset is split with stratification to preserve class proportions:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}, \quad |\mathcal{D}_{\text{test}}| = 0.2 \cdot |\mathcal{D}|$$

Stratified splitting ensures that rare provinces are represented in both splits.

B. Neural Network

Given a property listing with features $\mathbf{x} \in \mathbb{R}^d$, the goal is to predict which of the 9 South African provinces it belongs to. This is a multi-class classification problem.

The model is a fully connected feedforward neural network with batch normalization and dropout.

1) Layer Structure:

$$\mathbb{R}^d \xrightarrow{FC_1} \mathbb{R}^{256} \xrightarrow{FC_2} \mathbb{R}^{128} \xrightarrow{FC_3} \mathbb{R}^{64} \xrightarrow{FC_4} \mathbb{R}^{32} \xrightarrow{FC_5} \mathbb{R}^9$$

Layer	Operation	Output Dim	Act
1	Linear → BN → ReLU → Dropout	256	ReLU
2	Linear → BN → ReLU → Dropout	128	ReLU
3	Linear → BN → ReLU → Dropout	64	ReLU
4	Linear → BN → ReLU → Dropout	32	ReLU
5	Linear	9	None

2) Training Details:

- **Batch normalization**: Applied after each linear layer (except output).
- **Dropout probability**: $p = 0.2$ for all dropout layers.
- **Loss function**: Cross-entropy.
- **Optimizer**: Adam with learning rate 10^{-3}.
- **L2 regularization (weight decay)**: $\lambda = 10^{-4}$.
- **Gradient clipping**: Global norm clipped to $\tau = 1.0$.
- **Training**: Mini-batch size 32, early stopping based on validation loss.

At inference, the predicted class is obtained as the argmax of the logits, and probabilities are computed via softmax.

C. Random Forest

A Random Forest classifier is trained to predict the South African province of a property listing from its normalized features. The input to the model is the normalized feature matrix.

Labels are integer-encoded as described in Section VI. The data is split with stratification as above.

1) Implementation Parameters:

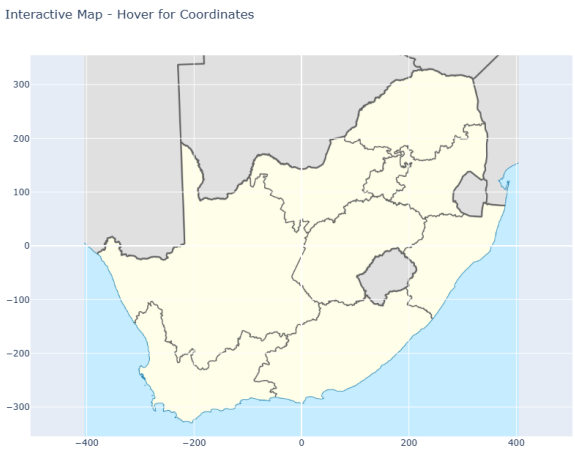
- **Number of trees**: $B = 200$.
- **Split criterion**: Gini impurity.
- **Maximum depth**: Unlimited (nodes expanded until pure or min_samples_split reached).
- **Minimum samples split**: 2.
- **Minimum samples leaf**: 1.
- **Max features**: $\lfloor \sqrt{d} \rfloor$ at each split.
- **Bootstrap**: Enabled.
- **Out-of-bag (OOB) score**: Computed for internal validation.

Prediction is the majority vote of all trees.

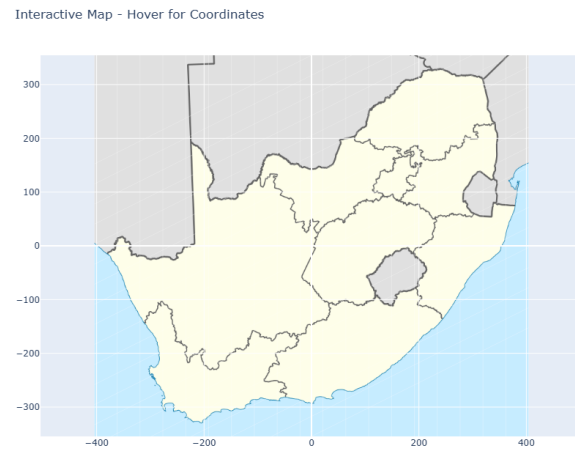
D. Support Vector Machine

A Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel is trained to predict the South African province of a property listing. SVMs are maximum-margin classifiers.

Unlike tree-based models, SVMs are sensitive to feature scale. The normalized data is therefore re-scaled using a StandardScaler fitted on the training set only before training.

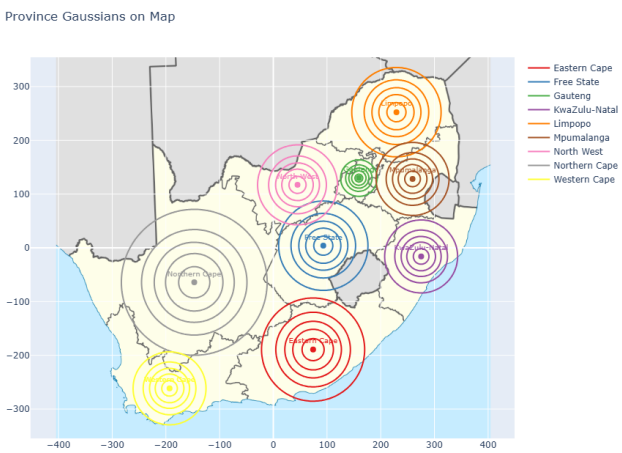


(a) South Africa in Euclidean Space

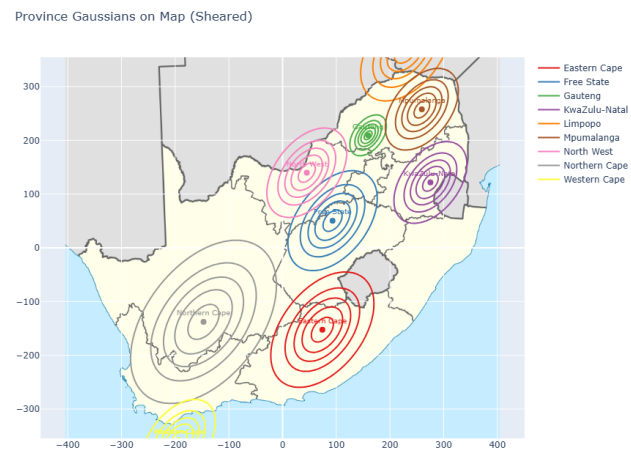


(b) South Africa in Transformed Space

Fig. 1: Illustrates coordinate systems before and after transformation



(a) Province Gaussian



(b) Province Gaussian after Basis Change

Fig. 2: Illustrates how the Gaussian changes after basis change

1) Implementation Parameters:

- **Kernel**: Radial Basis Function (RBF).
- **Regularization parameter**: $C = 1.0$.
- **Gamma**: Set to `'scale'` $= 1/(d \cdot \mathrm{Var}(X))$.
- **Multi-class strategy**: One-vs-One (OvO), yielding 36 binary classifiers.
- **Probability calibration**: Enabled (Platt scaling) for ROC-AUC computation.

Decision function outputs are converted to class predictions via voting.

E. XGBoost

XGBoost (Extreme Gradient Boosting) is a scalable, regularised implementation of gradient boosted decision trees. Unlike Random Forest which builds trees in parallel via bagging, XGBoost builds trees sequentially — each new tree corrects the residual errors of all previous trees.

1) Implementation Parameters:

- **Number of boosting rounds**: $B = 300$.
- **Objective**: `multi:softprob` (softmax probability output).
- **Eval metric**: Multiclass log-loss (`mlogloss`).
- **Learning rate (eta)**: $\eta = 0.05$.
- **Maximum tree depth**: 6.
- **Subsample ratio**: 0.8 (80% of training rows used per tree, without replacement).
- **Column subsample per tree**: 0.8 (80% of features used per tree).
- **Minimum child weight**: 1 (minimum sum of instance weights in a child).

TABLE I: Performance Comparison: Neural Network vs Support Vector Machine

Metric	NN	SVM
Accuracy	0.9502	0.9733
Weighted Precision	0.9328	0.9730
Weighted Recall	0.9502	0.9733
Weighted F1-Score	0.9400	0.9731

TABLE II: Performance Comparison: XGBoost vs Random Forest

Metric	XGBoost	RF
Accuracy	0.98	0.98
Macro Precision	0.97	0.97
Macro Recall	0.97	0.97
Macro F1-Score	0.97	0.97
Weighted Precision	0.98	0.98
Weighted Recall	0.98	0.98
Weighted F1-Score	0.98	0.98

- **Gamma**: 0 (minimum loss reduction required for split).
- **Regularization parameters**: $\lambda = 1$ (L2 on leaf weights), $\alpha = 0$ (L1).

XGBoost builds $9 \times 300 = 2700$ trees (one per class per round). Final prediction is the class with highest softmax probability.

VII. Discussion

The initial experimental results revealed a fundamental limitation in the predictive power of the feature set. When provided with only the basic property attributes (price, bedrooms, bathrooms, square meterage, and garage count), all models failed to achieve consistent performance, with accuracy remaining below 50% and F1-scores below 40%, despite extensive hyperparameter tuning and variations in model complexity.

This poor performance stems from the inherent nature of the prediction task. The models were effectively attempting to learn a price-per-square-meter mapping for each province. However, provinces with similar price-per-square-meter distributions become indistinguishable when only these features are available. This suggests a data-centric issue rather than a model-centric one—the feature space lacks sufficient discriminatory information to reliably differentiate between provinces with comparable property price structures.

To address this limitation, we explicitly engineered the price-per-square-meter ratio as an input feature. While this provided some improvement, the fundamental ambiguity between provinces with similar pricing patterns persisted, confirming that additional information beyond price-based features is necessary for accurate geographic classification.

The introduction of positional encoding through Gaussian-distributed coordinate sampling dramatically altered model behaviour. With this augmented feature set, all models achieved near-perfect classification performance, as shown in Tables I and II. The neural network achieved 95.02% accuracy while the SVM reached 97.33% (Table I), and both XGBoost and random forest attained 98% accuracy across all metrics (Table II).

However, this success came with a significant caveat: the models exhibited strong overfitting to the positional encoding itself. The classification task became too trivial—the models learned to associate specific coordinate regions with provinces rather than learning meaningful relationships between property attributes and geographic location. This overfitting manifests in the near-identical performance metrics across all model architectures, as evidenced by the XGBoost and random forest results in Table II, where both models achieve precisely the same scores. This uniformity suggests that the positional features dominate the learning signal and overshadow the property attributes entirely, making the specific choice of model architecture largely irrelevant.

The comparison between neural network and SVM in Table I reveals that SVM slightly outperforms the neural network across all metrics, with a 2.31% higher accuracy and approximately 3.3% higher weighted F1-score. This suggests that the maximum-margin classification approach of SVMs may be particularly well-suited to the decision boundaries created by the positional encoding.

These findings highlight a critical trade-off in feature engineering for geographic classification tasks. While spatial features can provide powerful discriminatory information, they must be carefully balanced to ensure models learn meaningful representations rather than simply memorizing location-specific patterns. Future work should explore more sophisticated spatial representations that encode geographic relationships without providing deterministic location information, such as hierarchical spatial embeddings or noise-injected coordinate systems.

VIII. Conclusion

This paper presented a comprehensive comparative analysis of four machine learning models—deep neural networks, random forests, support vector machines, and XGBoost—applied to the task of classifying South African property listings by province of origin using data scraped from Property24.

Our investigation revealed several important findings. First, property attributes alone (price, bedrooms, bathrooms, size, garage count) are insufficient for reliable geographic classification, with all models failing to exceed 50% accuracy and 40% F1-scores despite extensive hyperparameter optimization. This limitation stems from the fundamental ambiguity of price-based feature spaces: provinces with similar price-per-square-meter distributions become statistically indistinguishable when only these features are available.

Second, we demonstrated that the introduction of positional encoding through Gaussian-distributed coordinate sampling dramatically improves classification performance, with all models achieving near-perfect accuracy (95–98%), as detailed in Tables I and II. However, this performance gain comes with a significant caveat: models exhibit strong overfitting to the positional features, learning to associate specific coordinate regions with provinces rather than discovering meaningful

relationships between property attributes and geographic location. The near-identical performance across all model architectures, particularly the identical scores for XGBoost and random forest in Table II, confirms that the positional features dominate the learning signal entirely.

Third, our comparative analysis revealed that when sufficient discriminatory information is available (as in the positionally encoded features), all four model classes perform similarly, suggesting that feature engineering—rather than model selection—is the critical determinant of success in this domain. The SVM achieved slightly higher performance (97.33% accuracy) compared to the neural network (95.02% accuracy) in the comparison shown in Table I, while XGBoost and random forest performed identically (98% accuracy across all metrics) with the augmented feature set, as shown in Table II.

These findings have important implications for geographic classification tasks in real estate and beyond. High classification accuracy achieved through spatial features may mask underlying failures in representational learning, producing models that cannot generalize to new geographic regions or adapt to changing property market conditions. Practitioners must carefully balance the discriminatory power of spatial features against the risk of overfitting to location-specific patterns.

In conclusion, this work demonstrates both the potential and the pitfalls of geographic classification in real estate analytics. While positional encoding offers a powerful mechanism for incorporating spatial information, careful consideration must be given to how such features shape model behavior and what representations models actually learn. The path forward lies not in maximising accuracy at any cost, but in developing feature representations and model architectures that capture meaningful geographic relationships while maintaining the ability to generalise beyond the training distribution.